COMPUTER NETWORKS

ASSIGNMENT 2 – CO2

CO2 Coverage: Configure IP addressing schemes, subnetting and basic routing techniques using a simulation tool, for efficient network design and topology – packet switching, IPv4 addressing, subnetting, distance-vector and link-state routing, multicast, ICMP, IPv6, software-defined networking, network virtualisation and OpenFlow.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Simulation Tool: Cisco Packet Tracer (or GNS3)
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task asks you to calculate or analyse first, and then judge, decide or justify)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA07-<RegisterNumber>-Assignment2** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the Python programs, Packet Tracer (.pkt) files, device configurations, Wireshark captures, command outputs and screenshots.
- Include a README.md with your name, register number, the operating system, Python version and Packet Tracer (or NS-3) version used, the steps to run each program or open each simulation, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given program or device configuration into your repository, replace every TODO with working code or commands without changing the rest, run it (Python 3 or Cisco Packet Tracer) and compare your output with the expected output or result.

Alternative Concept Analysis: Describe how the solution would change if the alternative concept were used, and evaluate both solutions using network measures such as throughput, delay, jitter, packet loss, overhead, reliability, security and cost, supported by your own calculations, simulations or measurements wherever possible.

Q1 – Spot the Invalid Addresses — 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Address Clashes in a Hospital Group

A hospital group's IT team has collected the addresses configured on some devices. Several devices cannot communicate, and one clinic says it has "run out of addresses".

```
Admission PC     172.16.45.200 /20      gateway 172.16.32.1
Lab printer      10.0.15.255   /21      gateway 10.0.8.1
X-ray server     192.168.4.0   /23      gateway 192.168.4.1
Clinic router    203.0.113.10  /29      (public, from the internet provider)
```

Your Challenge

For each device, find the network address, broadcast address and usable host range, and decide whether the address is a valid host address. Two of them contain a trap.

Inquiry

The clinic's 18 devices share the one public /29 address block through NAT. Show how the NAT table lets two PCs both use source port 50000 at the same time. Then explain why packets of the same file can arrive in a different order (datagram switching), and what would be different with virtual circuits.

Alternative Concept Analysis

Suppose the clinic were given a public /27 instead of using NAT behind a /29. Count the public addresses needed for 18 devices, state the cost and exposure to the internet, and identify which application problems NAT would no longer cause.

Evidence Expected

Correct network, broadcast and host ranges, the two traps identified with reasons, a NAT table example and a reasoned comparison of datagram and virtual-circuit switching.

Q2 – Subnet a Campus That Keeps Growing — 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Engineering College Campus

A college has 192.168.10.0/24 for laboratories (100 hosts), staff (50 hosts), the library (25 hosts) and a point-to-point WAN link (2 hosts).

Your Challenge

Design the subnets twice – with equal-size subnets and with VLSM – giving network, prefix, mask, first and last host and broadcast for each. Then answer the trap: six months later the college adds a hostel network of 60 hosts. Can it be fitted into the remaining space of your VLSM plan without renumbering? If not, what is the smallest change that makes it fit?

Route Summarisation

The city campus uses 172.16.8.0/24, 172.16.9.0/24, 172.16.10.0/24 and 172.16.11.0/24. Find the single summary route, and show what goes wrong if 172.16.12.0/24 is added later and someone summarises it as 172.16.8.0/21.

Complete the Code

Complete TODO 1 below, and run the program with Python 3.

```
"""Variable-length subnetting (VLSM) of 192.168.10.0/24 for a college campus."""
import math


def to_int(ip):
    a, b, c, d = map(int, ip.split("."))
    return (a << 24) | (b << 16) | (c << 8) | d


def to_ip(n):
    return ".".join(str((n >> s) & 255) for s in (24, 16, 8, 0))


def vlsm(base, needs):
    addr = to_int(base)
    for name, hosts in sorted(needs, key=lambda x: -x[1]):
        # TODO 1: Find the smallest block that holds hosts + 2 addresses (a power of
        #   two), its prefix length 32 - log2(block), and print the network, prefix,
        #   first and last usable host and broadcast address. Then move addr to the next
        #   block
        pass  # replace this line with your code
    return to_ip(addr)


needs = [("Labs", 100), ("Staff", 50), ("Library", 25), ("WAN link", 2)]
print("next free address:", vlsm("192.168.10.0", needs))
```

Expected output:

```
Labs          192.168.10.0/25  hosts 192.168.10.1 - 192.168.10.126  bcast 192.168.10.127
Staff       192.168.10.128/26  hosts 192.168.10.129 - 192.168.10.190  bcast 192.168.10.191
Library     192.168.10.192/27  hosts 192.168.10.193 - 192.168.10.222  bcast 192.168.10.223
WAN link    192.168.10.224/30  hosts 192.168.10.225 - 192.168.10.226  bcast 192.168.10.227
next free address: 192.168.10.228
```

Check every subnet with Python’s ipaddress module, then add the 60-host hostel network and see what happens.

Alternative Concept Analysis

Redo the college plan starting from the smallest subnet instead of the largest. Show where the allocation goes wrong (overlapping or misaligned blocks) and explain why VLSM must assign the largest subnets first.

Evidence Expected

Complete subnet tables for both methods, a correct answer to the hostel trap, a correct summary route and an explained summarisation error, verified in Packet Tracer.

Q3 – Which Way Will the Packet Go? 6 Marks

Bloom’s Level: K5 – Evaluate

Problem Context – Two Campuses and a Backup Link

The main-campus router has this routing table:

```
S    10.0.0.0/8         via 192.0.2.1
S    10.1.0.0/16        via 192.0.2.5
S    10.1.2.0/24        via 192.0.2.9
S    10.1.2.128/25      via 192.0.2.13
S*   0.0.0.0/0          via 198.51.100.1
```

Your Challenge

Determine the next hop for packets to 10.1.2.200, 10.1.2.100, 10.1.9.9, 10.7.7.7 and 8.8.8.8, and state the rule you used. Then build the two-campus network in Packet Tracer with static routes, add a backup link with a floating static route (administrative distance 200), and show that traffic moves to the backup when the main link is shut down.

Complete the Code

Complete TODO 1 and TODO 2 in the router configuration below, and enter it on R1 in Cisco Packet Tracer. Configure R2 in the same way with the reverse route.

```
! Cisco Packet Tracer: static routing between the Main campus (R1) and City campus (R2).
! R1 LAN 192.168.10.0/24 on G0/0, R2 LAN 192.168.20.0/24, serial link 10.0.0.0/30.
enable
configure terminal
hostname R1
interface GigabitEthernet0/0
 ip address 192.168.10.1 255.255.255.0
 no shutdown
! TODO 1: Configure Serial0/0/0 with 10.0.0.1/30, set the clock rate to 64000 (R1 is the
!   DCE end) and bring it up

exit
! TODO 2: Add a static route to the City campus LAN 192.168.20.0/24 through the next hop
!   10.0.0.2

end
write memory
show ip route
```

Expected result: show ip route on R1 lists 192.168.20.0/24 as a static route (S) through 10.0.0.2, and a PC at the main campus can ping a PC at the city campus. Remove the clock rate on the DCE end and record what happens to the serial link.

Alternative Concept Analysis

Replace the five static routes with OSPF on both campuses. Count the configuration lines needed for 2 campuses and for 5 campuses in each approach, and compare the time to recover when the main link fails.

Evidence Expected

Correct next hops with the longest-prefix rule, a working Packet Tracer network with a floating static route, and before-and-after routing tables.

Q4 – Watch Distance-Vector Routing Count to Infinity — 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – District Government Offices

Five routers connect government offices with the link costs A–B 2, A–C 7, B–C 3, B–D 8, C–D 1, D–E 4.

Your Challenge

- Build every router's distance table round by round until nothing changes, and state how many rounds it took
- Explain why A's best route to C is not the direct link

- Now take three routers in a line X–Y–Z (cost 1 each). The link Y–Z fails. Show, round by round, how X and Y keep raising their distance to Z (count to infinity), and how split horizon with poison reverse stops it
- Configure RIP in Packet Tracer and observe the convergence after a link failure

Complete the Code

Complete TODO 1 below, and run the program with Python 3.

```
"""Distance-vector routing (Bellman-Ford exchange) among five campus routers."""
INF = 99
names = "ABCDE"
cost = {("A", "B"): 2, ("A", "C"): 7, ("B", "C"): 3, ("B", "D"): 8,
        ("C", "D"): 1, ("D", "E"): 4}
link = {}
for (u, v), c in cost.items():
    link[u, v] = link[v, u] = c

table = {u: {v: (0 if u == v else link.get((u, v), INF)) for v in names} for u in names}

rounds = 0
changed = True
while changed:
    changed = False
    rounds += 1
    old = {u: dict(table[u]) for u in names}       # vectors received this round
    for u in names:
        for n in names:
            if (u, n) not in link:
                continue
            for dest in names:
                # TODO 1: Bellman-Ford update: if going through neighbour n (link cost +
                #   n's old distance to dest) is cheaper than table[u][dest], update it
                #   and set changed = True
                pass  # replace this line with your code
print(f"converged after {rounds} rounds")
for u in names:
    print(u, " ".join(f"{table[u][v]:2d}" for v in names))
```

Expected output:

```
converged after 4 rounds
A  0  2  5  6 10
B  2  0  3  4  8
C  5  3  0  1  5
D  6  4  1  0  4
E 10  8  5  4  0
```

After convergence, set the cost of link C–D to 99 (failed) and rerun; record the new tables and rounds.

Alternative Concept Analysis

Run the same five-router network with link-state routing instead. After the C–D link fails, compare how many messages each method sends and how many rounds pass before every router has a correct route.

Evidence Expected

Correct round-by-round tables, a traced count-to-infinity example, the effect of poison reverse and RIP evidence from Packet Tracer.

Q5 – Why Does OSPF Treat 1 Gbps and 10 Gbps the Same? 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – A State Internet Service Provider

An internet service provider connects cities with 10 Mbps, 100 Mbps, 1 Gbps and 10 Gbps links. After moving to OSPF, traffic still uses a 1 Gbps path instead of a parallel 10 Gbps path.

Your Challenge

- Calculate the OSPF cost of each link speed with the default reference bandwidth of 100 Mbps, and explain the problem
- Choose a new reference bandwidth, recalculate the costs, and run Dijkstra's algorithm by hand from one router on a 6-router network of your design
- Configure OSPF in Packet Tracer (with auto-cost reference-bandwidth) and show that the 10 Gbps path is now chosen
- Decide whether the provider should use one OSPF area or several, and justify the decision

Alternative Concept Analysis

The provider could use EIGRP (which uses bandwidth and delay in its metric) or IS-IS instead of OSPF. Compute the metric each would give the 1 Gbps and 10 Gbps paths, and decide which protocol avoids the problem without changing the reference bandwidth.

Evidence Expected

Correct cost calculations, a hand-worked shortest-path tree, Packet Tracer evidence and a justified area design.

Q6 – IPv6, ICMP, Multicast and SDN in a Smart City 5 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Modernising a Smart-City Network

A smart city will connect 2 million sensors, send each traffic-camera stream to several control rooms, and wants to change routing centrally.

Your Challenge

- Write 2001:0db8:0000:0000:0000:ff00:0042:8329 in its shortest form, and build the EUI-64 interface identifier for a sensor with MAC address 00:1A:2B:3C:4D:5E
- Explain, using the TTL field, how traceroute uses ICMP Time Exceeded messages to discover each router
- Find the multicast MAC address for the group 239.1.2.3, and explain why one camera stream reaches several control rooms without being copied by the camera
- An OpenFlow switch has the rules (priority 200: dst 10.5.0.0/16 → port 3), (priority 100: TCP port 80 → controller), (priority 0: drop). Decide what happens to an HTTP packet to 10.5.1.1 and to an SSH packet to 10.6.1.1, and judge whether SDN is worth adopting for the city

Alternative Concept Analysis

Compare moving straight to IPv6 with running dual-stack or tunnelling IPv6 over IPv4 for the city. Calculate the extra header bytes of a 6in4 tunnel on a 100-byte sensor packet, and decide which approach the city should start with.

Evidence Expected

Correct IPv6, EUI-64 and multicast calculations, a correct traceroute explanation and a reasoned SDN decision.

DAY-4 DESIGN CHALLENGE

Q7 – Find the Weak Point, Redesign and Validate 13 Marks

Bloom's Level: K5 – Evaluate

Review the evidence you generated in Q1–Q6. Select **ONE** network that shows a significant limitation as it grows or as the requirement changes.

You are **not** permitted simply to add more equipment and claim the network is better. You must understand the limitation first and then develop an appropriate redesign.

A. Identify the Limitation – 2 Marks

Using your own evidence, show where the design fails or slows down and which part of the addressing or routing design causes it.

Evidence Expected

A precise analysis supported by evidence.

B. Define the Design Objective – 2 Marks

State a measurable objective (for example: convergence time, addresses wasted, routing-table size).

Evidence Expected

A clear, measurable objective.

C. Design the Revised Network – 4 Marks

Explain what you have changed and any new trade-off. Provide the topology diagram and address table.

Evidence Expected

A coherent redesign directly addressing the limitation.

D. Predict Before Implementing – 1 Mark

Predict the improvement and the measure in which you expect it.

Evidence Expected

A clear, testable prediction.

E. Implement and Validate – 2 Marks

Compare **Original → Redesigned** in Packet Tracer using the same tests.

Evidence Expected

Comparable evidence for both versions.

F. Defend the Final Design – 2 Marks

Judge whether the objective was achieved, the new costs, and where the original may still be preferable.

Evidence Expected

A critical, evidence-based conclusion.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	IPv4 addressing, NAT and packet switching	K5 – Evaluate	6
Q2	Subnetting, VLSM and summarisation	K5 – Evaluate	8
Q3	Longest-prefix match and static routing	K5 – Evaluate	6
Q4	Distance-vector routing	K5 – Evaluate	6
Q5	Link-state routing (OSPF)	K5 – Evaluate	6
Q6	IPv6, ICMP, multicast and SDN	K5 – Evaluate	5
Q7	Day-4 Design Challenge – Redesign & Validate	K5 – Evaluate	13
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your assignment report containing the required evidence for Q1–Q7
- Packet Tracer (.pkt) files, router configurations and Python programs
- Address tables, routing tables, ping and traceroute outputs and screenshots

Assessment Verification

Students may be given a new address or routing table and asked to solve it on the spot during a brief verification discussion/viva.

Overall Assessment Expectation

Calculate → Configure → Simulate → Test → Analyse → Redesign → Validate

COMPUTER NETWORKS (CSA07) – ASSIGNMENT 2 (CO2)

IP Addressing, Subnetting and Routing – Answer Sheet

Name	AB Sai Saranya	Register No.	1921260020
Course	CSA07 – Computer Networks	Total marks	50
GitHub repository	CSA07-1921260020-Assignment2	Tools used	Python 3, Cisco Packet Tracer

Q1 – Spot the Invalid Addresses (6 marks)

QUESTION
Hospital devices: Admission PC 172.16.45.200/20 gw 172.16.32.1; Lab printer 10.0.15.255/21 gw 10.0.8.1; X-ray server 192.168.4.0/23 gw 192.168.4.1; Clinic router 203.0.113.10/29 (public). For each find network, broadcast, usable range and whether it is a valid host; two contain a trap. **Inquiry:** 18 clinic devices share a /29 through NAT – show the NAT table letting two PCs both use source port 50000; why packets of one file can arrive out of order (datagram switching) and what virtual circuits change. **Alternative:** a public /27 instead of NAT behind a /29.

Device	Mask / block	Network	Broadcast	Usable range	Valid host?
Admission PC 172.16.45.200/20	255.255.240.0; 16 in 3rd octet (45 in 32–47)	172.16.32.0	172.16.47.255	172.16.32.1 – 172.16.47.254	**Valid**; gateway .32.1 is in the same subnet
Lab printer 10.0.15.255/21	255.255.248.0; 8 in 3rd octet (15 in 8–15)	10.0.8.0	10.0.15.255	10.0.8.1 – 10.0.15.254	**INVALID – TRAP 1:** it is the subnet's broadcast address
X-ray server 192.168.4.0/23	255.255.254.0; 2 in 3rd octet (4 in 4–5)	192.168.4.0	192.168.5.255	192.168.4.1 – 192.168.5.254	**INVALID – TRAP 2:** it is the network address (gateway .4.1 is fine)
Clinic router 203.0.113.10/29	255.255.255.248; block of 8	203.0.113.8	203.0.113.15	203.0.113.9 – .14 (6 hosts)	**Valid** (public)

'Run out of addresses': a /29 gives only 6 usable hosts, but the clinic has 18 devices, hence NAT/PAT.

Inquiry – NAT table (PAT) with the same source port

Inside local (private)	Inside global (public)	Destination
192.168.1.10 : 50000	203.0.113.10 : 61001	93.184.216.34 : 443
192.168.1.11 : 50000	203.0.113.10 : 61002	93.184.216.34 : 443

Both PCs use source port 50000 at the same time because the router rewrites the source to the same public IP but a **different public port** (61001, 61002) and uses the (public port ↔ private IP:port) entry to translate each reply back to the right PC.

- **Datagram switching:** every packet carries the destination address and each router chooses the next hop independently (routes change with load or failures), so the packets of one file may take different paths with different delays and arrive **out of order or lost**; TCP at the end hosts reorders and retransmits. No call set-up; robust to failure.
- **Virtual circuit:** a path is set up first, packets carry a short circuit number (VCI) and follow the same path in order, with per-circuit state in each switch, reserved resources and predictable delay; but set-up delay, state, and if a switch fails the circuit breaks and must be rebuilt.

Alternative – public /27 instead of NAT behind a /29

Addresses needed: 18 devices + 1 router interface = 19 hosts; a /28 has only 14 usable, so a **/27** (32 addresses, 30 usable) is the smallest block, leaving 11 spare. **Cost:** 30 public IPv4 addresses (scarce, billed by the ISP, often several times the /29 price). **Exposure:** every device is directly reachable from the internet, so a stateful firewall and patching become essential (NAT's incidental hiding is lost). **Problems removed:** end-to-end connectivity for inbound services and servers, no port-forwarding, no breakage of IPsec/VoIP/SIP/active-mode FTP/peer-to-peer caused by address translation, unambiguous logging of which device did what.

Evidence to attach: Python ipaddress verification and Packet Tracer NAT table (show ip nat translations).

Q2 – Subnet a Campus That Keeps Growing (8 marks)

QUESTION
192.168.10.0/24 for laboratories (100 hosts), staff (50), library (25) and a point-to-point WAN link (2). Design subnets twice – equal-size and VLSM – with network, prefix, mask, first/last host and broadcast. Trap: six months later a hostel of 60 hosts is added – can it fit in the remaining VLSM space without renumbering? If not, the smallest change. **Route summarisation:** 172.16.8.0/24 to 172.16.11.0/24 – single summary route; what goes wrong if 172.16.12.0/24 is added and summarised as 172.16.8.0/21. **Complete the Code** (TODO 1). **Alternative:** allocate from the smallest subnet first.

Method 1 – equal-size subnets

Four subnets would need /26 (64 addresses, 62 hosts each), but the labs need 100 hosts > 62, so equal-size **fails** in one /24. Using /25 (126 hosts) for all makes 4 × 128 = 512 addresses, i.e. two /24 networks:

Subnet	Network/prefix	Mask	First host	Last host	Broadcast
Labs	192.168.10.0/25	255.255.255.128	192.168.10.1	192.168.10.126	192.168.10.127
Staff	192.168.10.128/25	255.255.255.128	192.168.10.129	192.168.10.254	192.168.10.255
Library	192.168.11.0/25	255.255.255.128	192.168.11.1	192.168.11.126	192.168.11.127
WAN link	192.168.11.128/25	255.255.255.128	192.168.11.129	192.168.11.254	192.168.11.255

Waste: 512 allocated for 177 hosts (65% unused); the WAN link wastes 126 of 128 addresses.

Method 2 – VLSM (largest first)

Subnet	Hosts	Block	Network/prefix	Mask	First host	Last host	Broadcast
Labs	100	128	192.168.10.0/25	255.255.255.128	192.168.10.1	192.168.10.126	192.168.10.127
Staff	50	64	192.168.10.128/26	255.255.255.192	192.168.10.129	192.168.10.190	192.168.10.191
Library	25	32	192.168.10.192/27	255.255.255.224	192.168.10.193	192.168.10.222	192.168.10.223
WAN link	2	4	192.168.10.224/30	255.255.255.252	192.168.10.225	192.168.10.226	192.168.10.227

Used 228 of 256 addresses; free: 192.168.10.228 – .255 (28 addresses).

Trap – the 60-host hostel

A hostel needs 60 + 2 = 62 → block of 64 (/26). Only 28 addresses remain, and the only /26 boundaries (.0, .64, .128, .192) are all in use. **It cannot be fitted without renumbering** – in fact the five blocks need 128 + 64 + 64 + 32 + 4 = 292 addresses, more than a /24 (256), so even renumbering inside the /24 cannot work (the program below shows the largest-first re-allocation pushing Staff and Library to new addresses). **Smallest change:** extend the campus address space to **192.168.10.0/23** and give the hostel **192.168.11.0/26** (hosts .1–.62, broadcast .63). Nothing existing is renumbered, and the whole campus still summarises as one /23.

Completed program (TODOs replaced; file: q2.py)

```
"""Variable-length subnetting (VLSM) of 192.168.10.0/24 for a college campus."""
import math, ipaddress

def to_int(ip):
    a, b, c, d = map(int, ip.split("."))
    return (a << 24) | (b << 16) | (c << 8) | d

def to_ip(n):
    return ".".join(str((n >> s) & 255) for s in (24, 16, 8, 0))

def vlsm(base, needs):
    addr = to_int(base)
    for name, hosts in sorted(needs, key=lambda x: -x[1]):
        block = 1 << math.ceil(math.log2(hosts + 2))
        prefix = 32 - int(math.log2(block))
        net = to_ip(addr)
        print(f"{name:<12}{net}/{prefix}  hosts {to_ip(addr + 1)} - {to_ip(addr + block - 2)}  bcast {to_ip(addr + block - 1)}")
        n = ipaddress.ip_network(f"{net}/{prefix}")
        assert str(n.network_address) == net          # alignment check
        addr += block
    return to_ip(addr)

needs = [("Labs", 100), ("Staff", 50), ("Library", 25), ("WAN link", 2)]
print("next free address:", vlsm("192.168.10.0", needs))
print("--- hostel (60 hosts) added:")
try:
    print("next free address:", vlsm("192.168.10.0", needs + [("Hostel", 60)]))
except AssertionError:
    print("misaligned block")
print("free tail left in /24 = 256 - 228 =", 256 - 228, "addresses; a /26 needs 64")
```

Output obtained (run with Python 3):

```
Labs        192.168.10.0/25  hosts 192.168.10.1 - 192.168.10.126  bcast 192.168.10.127
Staff       192.168.10.128/26  hosts 192.168.10.129 - 192.168.10.190  bcast 192.168.10.191
Library     192.168.10.192/27  hosts 192.168.10.193 - 192.168.10.222  bcast 192.168.10.223
WAN link    192.168.10.224/30  hosts 192.168.10.225 - 192.168.10.226  bcast 192.168.10.227
next free address: 192.168.10.228
--- hostel (60 hosts) added:
Labs        192.168.10.0/25  hosts 192.168.10.1 - 192.168.10.126  bcast 192.168.10.127
Hostel      192.168.10.128/26  hosts 192.168.10.129 - 192.168.10.190  bcast 192.168.10.191
Staff       192.168.10.192/26  hosts 192.168.10.193 - 192.168.10.254  bcast 192.168.10.255
Library     192.168.11.0/27  hosts 192.168.11.1 - 192.168.11.30  bcast 192.168.11.31
WAN link    192.168.11.32/30  hosts 192.168.11.33 - 192.168.11.34  bcast 192.168.11.35
next free address: 192.168.11.36
free tail left in /24 = 256 - 228 = 28 addresses; a /26 needs 64
```

Verification with Python's ipaddress module: the program asserts that every network address returned by `ipaddress.ip_network()` equals the address allocated (alignment) – all four subnets pass. (Packet Tracer: configure the four subnets on a router and ping across them.)

Route summarisation

Network	3rd octet (binary)
172.16.8.0/24	00001000
172.16.9.0/24	00001001
172.16.10.0/24	00001010
172.16.11.0/24	00001011
Common bits	000010xx → 22 bits

Summary route: 172.16.8.0/22 (mask 255.255.252.0, covers 172.16.8.0 – 172.16.11.255). If 172.16.12.0/24 is added and someone advertises **172.16.8.0/21** (covers 8.0 – 15.255), the summary also claims 172.16.13.0, .14.0 and .15.0 (not owned by this campus). Neighbours will send traffic for those networks to this campus, where it is dropped (black hole) or looped, and overlapping with another site that really owns 172.16.13–15 causes mis-routing. Correct: advertise 172.16.8.0/22 plus 172.16.12.0/24 separately (or reserve all of 172.16.12–15 for this campus before using /21).

Alternative – allocate from the smallest subnet first

Step	Allocation	Result
1	WAN /30 at 192.168.10.0 (0–3)	Correct
2	Library /27 must start on a multiple of 32; the next free address is .4 – a /27 starting at .4 is really network 192.168.10.0/27, which **overlaps the WAN block**	Overlap / misaligned block
3	Moving the library to .32/27, Staff to .64/26 and Labs to .128/25 works only by **wasting .4–.31 (28 addresses)** and leaves 0 free addresses	No growth space

Why largest first: blocks are powers of two that must begin at a multiple of their own size. Starting with the biggest keeps every later (smaller) block aligned automatically, leaves one contiguous free tail (.228–.255) for growth and wastes no gaps. Smallest-first misaligns larger blocks, creating overlaps or holes.

Evidence to attach: Packet Tracer topology with the VLSM addressing and ping results; show ip route.

Q3 – Which Way Will the Packet Go? (6 marks)

QUESTION

Main-campus routing table: S 10.0.0.0/8 via 192.0.2.1; S 10.1.0.0/16 via 192.0.2.5; S 10.1.2.0/24 via 192.0.2.9; S 10.1.2.128/25 via 192.0.2.13; S* 0.0.0.0/0 via 198.51.100.1. Determine the next hop for 10.1.2.200, 10.1.2.100, 10.1.9.9, 10.7.7.7, 8.8.8.8 and the rule. Build the two-campus network in Packet Tracer with static routes, add a floating static route (AD 200) and show failover. **Complete the Code** (TODO 1, 2) for R1 and R2. **Alternative:** OSPF instead of five static routes.

Destination	Matching routes	Longest match	Next hop
10.1.2.200	/8, /16, /24, /25 (.128–.255 includes 200)	10.1.2.128/25	**192.0.2.13**
10.1.2.100	/8, /16, /24 (100 < 128 so not /25)	10.1.2.0/24	**192.0.2.9**
10.1.9.9	/8, /16	10.1.0.0/16	**192.0.2.5**
10.7.7.7	/8 only	10.0.0.0/8	**192.0.2.1**
8.8.8.8	default only	0.0.0.0/0	**198.51.100.1**

Rule: longest-prefix match – the router chooses, among all routes matching the destination, the one with the most network bits (most specific); administrative distance only decides between routes of the same prefix.

R1 configuration (TODO 1 and TODO 2 completed)

```
enable
configure terminal
hostname R1
interface GigabitEthernet0/0
 ip address 192.168.10.1 255.255.255.0
 no shutdown
interface Serial0/0/0                      ! TODO 1
 ip address 10.0.0.1 255.255.255.252
 clock rate 64000                          ! R1 is the DCE end
 no shutdown
exit
ip route 192.168.20.0 255.255.255.0 10.0.0.2          ! TODO 2 (main route, AD 1)
ip route 192.168.20.0 255.255.255.0 10.0.1.2 200      ! floating static via backup link
end
write memory
show ip route
```

R2 configuration (reverse route)

```
hostname R2
interface GigabitEthernet0/0
 ip address 192.168.20.1 255.255.255.0
 no shutdown
interface Serial0/0/0
 ip address 10.0.0.2 255.255.255.252
 no shutdown
ip route 192.168.10.0 255.255.255.0 10.0.0.1
ip route 192.168.10.0 255.255.255.0 10.0.1.1 200
```

- **Expected show ip route on R1:** `S 192.168.20.0/24 [1/0] via 10.0.0.2` (the AD 200 route is not shown while the main route is up); PC1 at main campus pings PC2 at city campus.
- **Failover test:** `R1(config)# interface s0/0/0` → `shutdown`; the AD 1 route is removed, the floating route `S 192.168.20.0/24 [200/0] via 10.0.1.2` appears, and pings resume (first 1–2 packets may time out). Before: next hop 10.0.0.2; after: 10.0.1.2.
- **Remove the clock rate on the DCE end:** the serial interface shows up, line protocol is down (the DTE end has no clock, no keepalives), the link turns red in Packet Tracer, the connected route disappears and pings fail until `clock rate 64000` is re-entered.

Alternative – OSPF instead of static routes

	Static (+ floating backup)	OSPF
Configuration, 2 campuses	1 route per router + 1 floating = 4 lines	router ospf 1 + 2 network lines per router ≈ 6 lines
Configuration, 5 campuses (each LAN reachable from all)	4 remote LANs × 5 routers = 20 lines (40 with backups)	≈ 3–4 lines per router = 15–20 lines – does not grow with the number of destinations
Recovery after main link fails	Immediate if the interface goes down; **never** if the failure is beyond the next hop (no tracking)	Dead interval 40 s (default), ~1 s with fast hello/BFD; automatic, detects remote failures too

Decision: static + floating routes for two campuses; OSPF as soon as there are more than 3–4 routers.

Evidence to attach: Packet Tracer .pkt file, show ip route before and after shutdown, ping screenshots.

Q4 – Watch Distance-Vector Routing Count to Infinity (6 marks)

QUESTION
Five routers with costs A–B 2, A–C 7, B–C 3, B–D 8, C–D 1, D–E 4. Build every router's distance table round by round until nothing changes and state the number of rounds; why A's best route to C is not the direct link; X–Y–Z line (cost 1 each) where Y–Z fails – show count to infinity and how split horizon with poison reverse stops it; configure RIP in Packet Tracer. **Complete the Code** (TODO 1); set C–D to 99 and rerun. **Alternative:** link-state after C–D fails.

Distance tables round by round (columns A B C D E)

Round 0	A B C D E	Round 1	A B C D E	Round 2	A B C D E
A	0 2 7 ∞ ∞	A	0 2 5 8 ∞	A	0 2 5 6 12
B	2 0 3 8 ∞	B	2 0 3 4 12	B	2 0 3 4 8
C	7 3 0 1 ∞	C	5 3 0 1 5	C	5 3 0 1 5
D	∞ 8 1 0 4	D	8 4 1 0 4	D	6 4 1 0 4
E	∞ ∞ ∞ 4 0	E	∞ 12 5 4 0	E	12 8 5 4 0

Round 3	A B C D E	Round 4	A B C D E
A	0 2 5 6 10	A	0 2 5 6 10
B	2 0 3 4 8	B	2 0 3 4 8
C	5 3 0 1 5	C	5 3 0 1 5
D	6 4 1 0 4	D	6 4 1 0 4

Round 3	A B C D E	Round 4	A B C D E
E	10 8 5 4 0	E	10 8 5 4 0

Converged after **4 rounds** (rounds 1–3 change tables, round 4 detects no change). **A's best route to C:** direct cost 7, but A→B→C costs 2 + 3 = 5 < 7, so the 2-hop path is cheaper; Bellman-Ford chooses minimum cost, not minimum hops.

Completed program (TODOs replaced; file: q4.py)

```
"""Distance-vector routing (Bellman-Ford exchange) among five campus routers."""
import sys
INF = 99
names = "ABCDE"
def run(cost, show=False):
    link = {}
    for (u, v), c in cost.items():
        link[u, v] = link[v, u] = c
    table = {u: {v: (0 if u == v else link.get((u, v), INF)) for v in names} for u in names}
    rounds = 0
    changed = True
    while changed:
        changed = False
        rounds += 1
        old = {u: dict(table[u]) for u in names}
        for u in names:
            for n in names:
                if (u, n) not in link:
                    continue
                for dest in names:
                    if link[u, n] + old[n][dest] < table[u][dest]:
                        table[u][dest] = min(INF, link[u, n] + old[n][dest])
                        changed = True
        if show:
            print(f"-- after round {rounds}")
            for u in names:
                print(u, " ".join(f"{table[u][v]:2d}" for v in names))
    return rounds, table
cost = {("A","B"):2,("A","C"):7,("B","C"):3,("B","D"):8,("C","D"):1,("D","E"):4}
rounds, table = run(cost, show=True)
print(f"converged after {rounds} rounds")
for u in names:
    print(u, " ".join(f"{table[u][v]:2d}" for v in names))
print("=== C-D failed (cost 99), fresh start")
c2 = dict(cost); c2[("C","D")] = 99
rounds, table = run(c2)
print(f"converged after {rounds} rounds")
for u in names:
    print(u, " ".join(f"{table[u][v]:2d}" for v in names))
```

Output obtained (run with Python 3):

```
-- after round 1
A  0  2  5  8 99
B  2  0  3  4 12
C  5  3  0  1  5
D  8  4  1  0  4
E 99 12  5  4  0
-- after round 2
A  0  2  5  6 12
B  2  0  3  4  8
C  5  3  0  1  5
D  6  4  1  0  4
E 12  8  5  4  0
-- after round 3
A  0  2  5  6 10
B  2  0  3  4  8
C  5  3  0  1  5
D  6  4  1  0  4
E 10  8  5  4  0
-- after round 4
A  0  2  5  6 10
B  2  0  3  4  8
C  5  3  0  1  5
D  6  4  1  0  4
E 10  8  5  4  0
converged after 4 rounds
A  0  2  5  6 10
B  2  0  3  4  8
C  5  3  0  1  5
D  6  4  1  0  4
E 10  8  5  4  0
=== C-D failed (cost 99), fresh start
converged after 3 rounds
A  0  2  5 10 14
B  2  0  3  8 12
C  5  3  0 11 15
D 10  8 11  0  4
E 14 12 15  4  0
```

In the failure rerun (C–D cost 99) the tables become A→D 10, A→E 14, B→D 8, C→D 11, C→E 15: traffic from C to D now goes C→B→D (3+8 = 11), and the algorithm converges after **3 rounds** (see output above).

Count to infinity (X – Y – Z, each link cost 1)

Round	Event	Y's distance to Z	X's distance to Z
0	Before failure	1	2
1	Y–Z fails; Y believes X's advertised 2 → Y = 2 + 1	3	2
2	X hears Y = 3 → X = 4	3	4
3	Y hears X = 4 → Y = 5	5	4
4	X hears Y = 5 → X = 6	5	6
…	Continues +2 per exchange until the 'infinity' value (RIP = 16)	→ 16	→ 16

Poison reverse: X's route to Z goes through Y, so X advertises Z = ∞ (16) to Y. When Y–Z fails, Y gets no false alternative and marks Z unreachable immediately; X learns Z = ∞ in the next update, so there is no loop and no counting. (Split horizon simply omits the route; poison reverse sends it as ∞, which clears a bad route faster. Larger loops of 3+ routers still need hold-down/triggered updates.)

- **RIP in Packet Tracer:** `router rip`, `version 2`, `network a.b.c.d` for each connected network, `no auto-summary`; then shut a link and observe the route going possibly down then removed (invalid 180 s, flush 240 s, much faster with triggered updates), and the alternative route being learned.

Alternative – link-state after C–D fails (5 remaining links, 5 routers): C and D each flood one LSA; each router forwards a new LSA out of all interfaces except the incoming one: ≈ 2E − (N − 1) = 2×5 − 4 = 6 transmissions per LSA × 2 LSAs = **≈ 12 messages**, reaching every router in at most **3 flooding hops** (C→B→D→E), after which each router runs Dijkstra locally – every router correct in about 3 rounds (+ SPF delay). Distance vector after the failure: 3 rounds × 10 directed vector exchanges = **≈ 30 messages**, each carrying the whole vector, and with a count-to-infinity risk in other topologies. Link-state therefore converges in a comparable or fewer rounds with fewer messages, at the cost of more memory/CPU and a full topology database.

Evidence to attach: Python output before/after failure; RIP show ip route before/after in Packet Tracer.

Q5 – Why Does OSPF Treat 1 Gbps and 10 Gbps the Same? (6 marks)

QUESTION
ISP links of 10 Mbps, 100 Mbps, 1 Gbps and 10 Gbps; after moving to OSPF traffic still uses a 1 Gbps path instead of a parallel 10 Gbps path. Calculate OSPF cost of each speed with default reference 100 Mbps; choose a new reference bandwidth, recalculate, and run Dijkstra by hand on a 6-router network of your design; configure OSPF in Packet Tracer with auto-cost reference-bandwidth; decide one area or several. **Alternative:** EIGRP or IS-IS – compute the metric for 1 Gbps and 10 Gbps paths.

Link speed	Cost (reference 100 Mbps)	Cost (reference 100,000 Mbps)
10 Mbps	100/10 = **10**	100,000/10 = **10,000**
100 Mbps	100/100 = **1**	**1,000**
1 Gbps	100/1000 = 0.1 → **1** (minimum)	100,000/1,000 = **100**
10 Gbps	100/10,000 → **1**	100,000/10,000 = **10**

The problem: OSPF costs are whole numbers with a minimum of 1, so with the default 100 Mbps reference every link of 100 Mbps and above costs 1. A 1 Gbps and a 10 Gbps path of the same hop count are indistinguishable (a tie or an arbitrary choice) and OSPF may prefer a 1 Gbps path with fewer hops over a 10 Gbps path with more hops. **Fix:** set `auto-cost reference-bandwidth 100000` (100 Gbps) on all routers (so it is consistent and also leaves room for 40/100 G links).

Hand-worked Dijkstra from R1 (6-router design, new costs)

```
Links: R1-R2 10G (10)  R1-R3 1G (100)  R2-R3 1G (100)  R2-R4 10G (10)
       R3-R5 1G (100)  R4-R5 1G (100)  R4-R6 10G (10)  R5-R6 10G (10)
```

Step	Settled (cost)	Updated tentative costs
1	R1 (0)	R2 = 10, R3 = 100
2	R2 (10)	R3: 10+100 = 110 > 100 keep 100; R4 = 20
3	R4 (20)	R5 = 20+100 = 120; R6 = 30
4	R6 (30)	R5 = 30+10 = **40** (better than 120)
5	R5 (40)	R3: 40+100 = 140 > 100 keep
6	R3 (100)	–

Shortest-path tree from R1: R2 10, R4 20, R6 30, R5 40 via R1–R2–R4–R6–R5 (all 10G links), R3 100 (direct). Under the default costs R1→R5 would cost 2 via the 1G path R1–R3–R5 (instead of 4 hops over 10G) – that is the original problem; with the new costs the 10G path (40) beats the 1G path (200). (Verified with a Python Dijkstra.)

```
router ospf 1
 auto-cost reference-bandwidth 100000     ! same on all routers
 network 10.0.0.0 0.255.255.255 area 0
show ip ospf interface brief      ! check the cost of each interface
show ip route ospf                ! 10G path is now chosen
```

Area design: for the 6-router lab a single area (area 0) is enough. For the provider connecting many cities use **several areas** – a backbone area 0 plus one area per region – once the network is beyond about 30–50 routers: it limits LSA flooding and SPF runs to the area, gives smaller link-state databases, and allows summarisation at the area border routers. Cost: ABR design complexity and sub-optimal inter-area paths.

Alternative – EIGRP / IS-IS metric

Protocol	1 Gbps path (one link)	10 Gbps path (one link)	Distinguishes?
EIGRP (K1=K3=1): 256 × (10^7/BW_{kbps} + Σdelay/10 µs)	256 × (10 + 1) = **2,816**	256 × (1 + 1) = **512**	**Yes**
IS-IS (default metric 10 per link)	10	10	No (needs manual metrics)
OSPF (default reference)	1	1	No

EIGRP avoids the problem without changing any reference bandwidth (it uses bandwidth and delay; for rates above 10 Gbps the 'wide metrics' keep working). EIGRP is Cisco-oriented; OSPF and IS-IS are open standards, so the provider should choose OSPF with the corrected reference bandwidth unless the network is all-Cisco.

Evidence to attach: Packet Tracer OSPF topology; show ip ospf interface and show ip route before/after the change.

Q6 – IPv6, ICMP, Multicast and SDN in a Smart City (5 marks)

QUESTION
2 million sensors, camera streams to several control rooms, central routing change. Write 2001:0db8:0000:0000:0000:ff00:0042:8329 in shortest form and build the EUI-64 interface identifier for MAC 00:1A:2B:3C:4D:5E; explain traceroute with TTL and ICMP Time Exceeded; find the multicast MAC for 239.1.2.3 and why one camera stream reaches several control rooms; OpenFlow rules (priority 200 dst 10.5.0.0/16 → port 3; priority 100 TCP 80 → controller; priority 0 drop) – decide HTTP to 10.5.1.1 and SSH to 10.6.1.1 and judge SDN. **Alternative:** IPv6 vs dual-stack vs tunnelling – 6in4 overhead on a 100-byte packet.

- **Shortest form:** 2001:db8::ff00:42:8329 (drop leading zeros; the longest run of zero groups becomes ::).
- **EUI-64:** MAC 00:1A:2B:3C:4D:5E → insert FF:FE in the middle: 00:1A:2B:FF:FE:3C:4D:5E → flip the universal/local bit (7th bit of the first byte: 00 → 02) → interface ID **021A:2BFF:FE3C:4D5E** (full address e.g. 2001:db8:1::21a:2bff:fe3c:4d5e).
- **Traceroute:** the source sends probes with TTL = 1, 2, 3 …; each router decrements the TTL, and the router where it reaches 0 discards the packet and returns **ICMP Type 11 (Time Exceeded)** from its own address – revealing that hop and the round-trip time. The last probe reaches the destination, which answers ICMP Echo Reply / Port Unreachable, ending the trace.
- **Multicast MAC for 239.1.2.3:** 01:00:5e + lower 23 bits of the group address (1.2.3 → 01:02:03) = **01:00:5e:01:02:03**. The camera sends **one** stream to the group address; receivers (control rooms) join with IGMP; routers (PIM) and switches (IGMP snooping) replicate the packets only where there are receivers – the camera does no copying and each link carries one copy.
- **OpenFlow decisions:** HTTP packet to 10.5.1.1: matches rule 200 (dst 10.5.0.0/16) – the highest priority – so it is forwarded out of **port 3** (not sent to the controller, because the higher priority wins). SSH (TCP 22) to 10.6.1.1: not in 10.5.0.0/16, port is not 80, so only the priority-0 rule matches → **dropped**.
- **Is SDN worth it?** Yes for the city: central policy change in minutes instead of per-device CLI on thousands of devices, global view, traffic engineering, automation and virtual networks (OpenFlow/virtualisation). Risks: controller as single point of failure/attack (use a clustered controller), table-miss storms to the controller (as rule 100 would cause for all web traffic) and skills. Recommendation: start with a pilot in the core/data-centre and use a hybrid SDN design.

Alternative – IPv6-only vs dual-stack vs 6in4 tunnel

Approach	Extra header bytes	Pros	Cons
Native IPv6	none (40 B header vs 20 B IPv4)	Simplest, huge address space (2M sensors easily)	Existing IPv4 services need NAT64/DNS64
Dual stack	none (two stacks)	Works with both worlds, no tunnels	Double configuration and monitoring
6in4 tunnel	+20 B (IPv4 outer header): 100 → 120 B = **+20%**	Quick over IPv4-only links	MTU/fragmentation, extra delay, harder debugging

Decision: start with **dual stack** in the core and IPv6-only for the new sensors (with NAT64/DNS64 where IPv4 services are needed); use 6in4 tunnels only as a temporary bridge across IPv4-only links.

Evidence to attach: Packet Tracer ICMP/traceroute capture; address calculations.

Q7 – Find the Weak Point, Redesign and Validate (13 marks)

QUESTION
Review the evidence of Q1–Q6, select ONE network that shows a significant limitation as it grows or the requirement changes. Do not simply add equipment. A. Identify the limitation (2); B. Define a measurable design objective (2); C. Design the revised network with topology diagram and address table (4); D. Predict before implementing (1); E. Implement and validate Original → Redesigned in Packet Tracer with the same tests (2); F. Defend the final design (2).

Selected network: the engineering-college campus of Q2 (VLSM plan inside 192.168.10.0/24) with static routing to the city campus (Q3)

A. Identify the limitation (2 marks)

- **Addressing:** the VLSM plan uses 228 of 256 addresses (89%); the free tail is 28 addresses, but the new 60-host hostel needs a /26 – required space is 292 addresses > 256 (Q2). The plan cannot absorb growth without renumbering.
- **Routing:** every new subnet needs one more static route on every router: for 5 campuses 5 × 4 = 20 routes (Q3), and a failure beyond the next hop is never detected (no convergence at all). The addressing is also not summarisable: the four subnets of different sizes would be advertised as four separate routes.

B. Design objective (2 marks) – measurable

Metric	Original	Objective
Subnets that fit without renumbering	4 (hostel fails)	All 5 + 2 spare growth blocks, no renumbering
Address utilisation	228/256 = 89% allocated, 0 growth for a /26	≤ 60% of 512 used; ≥ 30% growth headroom per subnet
Routes advertised to the city campus	4 (one per subnet)	1 summary route
Static routes for 5 campuses	20 (+20 backup)	≤ 5 network statements (OSPF), 0 manual routes
Reconvergence after core-link failure	never for silent failures; ~instant only on carrier loss	≤ 5 s (OSPF fast hello 1 s / dead 4 s)

C. Revised network (4 marks)

Changes: (1) extend the address space to **192.168.10.0/23** (no existing subnet renumbered); (2) place the hostel in 192.168.11.0/26 and reserve growth blocks; (3) replace static routes with **OSPF area 0** and advertise the single summary 192.168.10.0/23 to the city campus; (4) keep a backup link with a floating route. **New trade-offs:** OSPF uses router CPU/memory and 1 s hello traffic (≈ a few kbps), needs an addressing/OSPF skill set, and the /23 consumes twice as many private addresses (which are cheap).

```
Topology:  [Labs SW]--+                              +--[Hostel SW]
           [Staff SW]--+--[R1 Main campus]==OSPF area 0==[R2 City campus]--[City LAN 172.16.8.0/22]
           [Library SW]-+       |  serial 10.0.0.0/30  |
                        +-- backup 10.0.1.0/30 (floating, AD 200)
```

Subnet	Network/prefix	Mask	Usable hosts	Gateway	Note
Labs	192.168.10.0/25	255.255.255.128	1–126	192.168.10.1	100 hosts, 26 spare
Staff	192.168.10.128/26	255.255.255.192	129–190	192.168.10.129	50 hosts, 12 spare
Library	192.168.10.192/27	255.255.255.224	193–222	192.168.10.193	25 hosts, 5 spare
WAN link	192.168.10.224/30	255.255.255.252	225–226	–	R1 .225 / R2 .226
Hostel (new)	192.168.11.0/26	255.255.255.192	1–62	192.168.11.1	60 hosts
Growth reserve	192.168.11.64/26 and 192.168.11.128/25	–	–	–	192 free addresses
Summary	**192.168.10.0/23**	255.255.254.0	–	–	1 route to city campus

```
router ospf 1
 network 192.168.10.0 0.0.1.255 area 0
 network 10.0.0.0 0.0.0.3 area 0
 hello/dead:  ip ospf hello-interval 1 / ip ospf dead-interval 4  (on the WAN interface)
```

D. Prediction (1 mark)

Predicted improvement: **(1)** the hostel (60 hosts) is reachable without renumbering any subnet (pings succeed, 5/5 subnets); **(2)** routes advertised to the city campus fall from 4 to **1** (show ip route on R2 shows 192.168.10.0/23 O); **(3)** static routes for 5 campuses fall from 20 to 5 network statements; **(4)** reconvergence after the main link fails drops to ≤ 5 s (≈ 4 s dead interval + SPF) with OSPF versus no recovery for a silent

failure with static routes. Measured with the same tests below.

E. Implement and validate (2 marks) – same tests on both versions

Test (Packet Tracer)	Original (static)	Redesigned (/23 + OSPF)	Measured result
Ping hostel PC to Labs PC	Hostel cannot be addressed without renumbering	Reply (predicted)	record screenshot
show ip route on R2 (campus routes)	4 static routes	1 OSPF summary route (predicted)	record screenshot
Shut main link, ping every 1 s	Fails (silent) or switches instantly only on carrier loss	≈ 4–5 lost pings then success (predicted)	record lost pings
Config lines (5 campuses)	20 + 20	≈ 15–20	count lines

Evidence to attach: Packet Tracer .pkt files for both versions and screenshots filling the 'Measured result' column (this was predicted from calculation; the simulation must be run by you in Packet Tracer).

F. Defend the final design (2 marks)

Objective achieved? By calculation – yes: 5 of 5 subnets fit (the original 4 of 5), the 4 routes become 1, address use is 292 of 512 = 57% (target ≤ 60%) and failover falls to ≈ 4 s. **New costs:** OSPF CPU/memory and 1 s hello traffic, more complex configuration, 256 more private addresses reserved, a skills requirement, and risk of a mis-advertised summary (as in the /21 error of Q2). **Where the original may still be preferable:** a two-router, very small campus with one link, no growth – static routes use no CPU, no protocol traffic and are simpler and more secure; or when routers do not support OSPF. **Overall:** the redesign removes the real limitation (address space and routing growth) rather than just adding equipment.